LLM-Driven TLA+ Specification of Essential Paxos

Ana Mako (6418643), Georgios Markozanis (6551254), Vuk Jurišić (5513847)

Team 3

1 Introduction

Consensus protocols are well known for being difficult to understand, specify, and verify correctly [1]. Even implementations that appear correct during testing may fail under rare execution schedules that are difficult to anticipate and reproduce. Because safety properties such as agreement must hold for every possible execution, testing alone provides limited assurance of correctness.

Formal specification and model checking offer a stronger alternative. TLA+ [2] provides a mathematical language for describing distributed systems and reasoning about their behaviour. Given a TLA+ specification, the TLC model checker can exhaustively explore all reachable states within a finite configuration and verify whether desired properties hold [3]. This exhaustive exploration is particularly attractive for consensus protocols, whose correctness relies on invariants that must hold universally rather than for a sampled subset of executions.

Despite its advantages, writing a correct TLA+ specification remains a challenge [2]. The developer must translate an informal protocol description into a precise state-machine model while simultaneously adhering to the syntactic and semantic requirements of TLA+. As a result, formal specification is often limited to researchers and engineers with expertise in both distributed systems and formal methods.

Recent advances in Large Language Models (LLMs) raise the possibility of lowering this barrier. Modern LLMs have demonstrated strong performance on code-generation and program-synthesis tasks [4], suggesting that they may also assist in translating protocol descriptions into formally verifiable specifications. Whether such models can reliably generate correct TLA+ specifications, however, remains an open question that is directly testable using SysMoBench [5], a benchmark for evaluating AI-generated TLA+ specifications. SysMoBench assesses generated specifications across four dimensions: syntactic correctness, successful model checking, conformance to execution traces, and satisfaction of expert-written safety invariants.

As a case study, we consider Essential Paxos [6], the minimal core implementation of Paxos contained in the cocagne/paxos library. Unlike the library's extended variants, which introduce additional mechanisms such as leader election, negative acknowledgements, and durable storage, Essential Paxos implements only the core consensus protocol. Its compact size and well-understood safety properties make it an attractive target for formal specification and verification.

Using Essential Paxos, we extend SysMoBench with a new benchmark task consisting of a custom instrumentation harness, execution traces, and expert-written safety invariants. We then use this benchmark to evaluate two LLM-based specification-generation strategies: a single-pass prompting approach and an iterative approach that incorporates feedback from failed verification attempts.

Research Question.

> *Does iterative, feedback-driven prompt refinement using SysMoBench evaluation signals produce a higher-quality TLA+ specification than a single carefully engineered prompt, and does this advantage hold across both Claude and GPT5.5?*

To answer this question, we conduct three experiments. Experiment 1 studies the iterative refinement process itself, recording how specifications evolve as verification feedback is incorporated into successive prompts. Experiment 2 compares the final prompts produced by each strategy across both models in clean sessions, allowing us to isolate the effect of prompt design from conversational history.

Experiment 3 evaluates the best-performing model–prompt combination under increasingly complex protocol configurations to understand how specification quality and verification cost scale with problem size.

2 Formal Modeling

2.1 Protocol Overview

Essential Paxos [6] implements single-decree consensus across three roles. *Proposers* initiate ballots, *Acceptors* vote on proposals, and *Learners* detect when a quorum has converged on a value. The protocol operates in two phases.

In Phase 1, a proposer selects a fresh ballot number n strictly greater than any it has used before and broadcasts PREPARE(n) to all acceptors. Each acceptor that has not already promised to a higher ballot responds with PROMISE(n, v_a, n_a), carrying the highest previously-accepted value v_a and its proposal ID n_a. An acceptor that has seen a higher ballot silently ignores the message.

In Phase 2, once a proposer has collected a quorum of promises, it selects a value v. If any promise carried a previously-accepted value, it must use the one with the highest ballot, otherwise it may propose its own value. The proposer then broadcasts ACCEPT(n, v). An acceptor that has not promoted its promise since Phase 1 records (n, v) and responds with ACCEPTED(n, v), otherwise it ignores the request. A learner declares consensus once a quorum of acceptors has sent ACCEPTED for the same ballot.

The five protocol actions in the reference implementation correspond directly to Python methods: `Proposer.prepare`, `Acceptor.recv_prepare`, `Proposer.recv_promise`, `Acceptor.recv_accept_request`, and `Learner.recv_accepted`.

Scope restriction. The cocagne/paxos library provides four additional modules beyond `essential.py`. `practical.py` extends the protocol with Prepare and Accept NACKs, a leadership-acquired callback, and a multi-instance resolution mechanism. `durable.py` adds crash-proof on-disk persistence. `functional.py` provides a stateless, functional-style wrapper. `external.py` exposes an externally-driven interface for custom transport layers. None of these are in scope for this work. Our TLA+ model captures only the state and transitions of `essential.py`: three role types (Proposer, Acceptor, Learner), two phases (Prepare/Promise and Accept/Accepted), and simple majority quorums.

2.2 TLA+ Specification Structure

The generated module `EssentialPaxos` declares the following constants and variables:

Listing 1: Constants and variables

```
CONSTANTS Acceptors, Proposers, Learners,
          Values, NoValue, MaxBallot
ProposalIDs == (0..MaxBallot) \X Proposers
```

State is stored in flat, per-role variables: `proposerProposalId`, `proposerValue`, `proposerPromisesRcvd`, `proposerLastAcceptedId`, `proposerNextNumber` (per-proposer ballot counter), `acceptorPromisedId`, `acceptorAcceptedId`, `acceptorAcceptedValue`, `learnerProposals`, `learnerAcceptors`, `learnerFinalValue`, `learnerFinalProposalId`, and a global message bag `msgs`.

The five protocol actions map directly to the Python event handlers:

Listing 2: Next predicate

```
Next ==
  \/ \E p \in Proposers    : Prepare(p)
  \/ \E a \in Acceptors, m : HandlePrepare(a,
    m)
  \/ \E p \in Proposers, m : HandlePromise(p,
    m)
  \/ \E a \in Acceptors, m : HandleAccept(a,
    m)
  \/ \E l \in Learners,  m :
    HandleAccepted(l, m)
```

2.3 Software Analysis Technique: TLA+ Model Checking with SysMoBench

Algorithm. The verification pipeline follows four sequentially dependent stages drawn from the SysMoBench benchmark [5]:

1. **Syntax correctness.** Statically check whether the generated system model uses valid TLA+ syntax using the SANY Syntactic Analyzer.

2. **Runtime correctness.** Check how much of the generated TLA+ can be executed using the TLC model checker, which is a proxy for logical self-consistency.

3. **Conformance.** Measure whether the model conforms to the system implementation via trace validation.

4. **Invariant correctness.** Model-check the system model against system-specific invariants that reflect the system's safety and liveness properties.

The canonical aggregate score weights these stages 0.15, 0.15, 0.35, and 0.35 respectively [5]. The heavy weight on the final two stages reflects that syntax and basic runtime correctness are necessary but insufficient conditions for a *faithful* specification.

Implementation architecture. SysMoBench is implemented as a Python package with a CLI entry point (`sysmobench`). Each task is declared by a `task.yaml` descriptor that names the source repository, source files, specModule, trace folder, and TV configuration (target actions, harness type, and a label-to-action map). The `direct_call` method submits a single structured prompt to the configured model and writes the generated `.tla` and `.cfg` files to a timestamped output directory. Subsequent stages are invoked sequentially, each receiving the output of the previous stage. A stage failure causes all later stages to be skipped, treating its score as zero in the aggregate.

The transition-validation stage is the most architecturally complex. Agents read the `tv-eval` SKILL.md guide, inspect the spec, and issue TLC queries for each trace window. This agentic loop is non-deterministic and may take 15–60 minutes per (task, model) cell, incurring approximately $1–4 in API cost.

Implementation limitations. The SysMoBench pipeline has several important limitations.

- **Claude Code script.** The main script for running the transition-validation stage is written for Claude Code agent. Any other agent running the TV spec has to do it manually through a skill and not using the premade script which deviates from the README.md instructions.

- **Trace coverage.** The harness runs four deterministic scenarios (happy path, dueling proposers, message loss, late message). Real deployments exercise a far richer set of concurrent-proposer races. The TV score reflects conformance to this narrow scenario set, not to all possible executions.

- **Agentic TV non-determinism.** The transition-validation step is itself performed by an LLM agent. Variability in the agent's variable-name translation can cause individual windows to be mis-classified even when the underlying spec is correct, artificially lowering the TV score.

Threats to validity. Three threats apply to our study.

- **Internal validity.** Iterative prompt refinement introduces a feedback loop. Each revision is informed by a specific SANY/TLC error, so the final prompt is overfitted to the errors encountered in our particular execution environment (Java version, TLC release, model configuration). A different environment might expose different errors and require different prompt constraints.

- **Construct validity.** The SysMoBench aggregate score measures only the four mechanically-checkable dimensions. A specification could score 100% while still being unsound in ways that fall outside the invariant template (e.g., modeling a subtle over-approximation that happens never to fire on the trace set).

- **External validity.** Essential Paxos is a single, educational protocol implementation scoped to 202 lines. Results on prompt engineering strategies may not generalize to more complex industrial protocols (e.g., Raft, CURP) that have larger source files, more message types, and richer invariant sets.

2.4 Prompt Design and Iterative Refinement

We used the SysMoBench `direct_call` method, which submits a single structured prompt contain-

ing a task description, a scope section, a network model description, output format requirements, and the full reference Python implementation (via the {source_code} template placeholder). A second prompt (phase2_config.txt) instructs the model to generate the companion .cfg file, and a third (phase3_invariant_implementation.txt) produces the EssentialPaxosInvariants module that concretizes the expert-written invariant templates against the generated spec's variable names. We have 2 final versions of all 3 prompts that were created by letting GPT5.5 and Claude agents iterate on the same starting prompt in order to get a TLA specification that passes as many steps as possible. Results are shown and discussed in Section 4.

3 Evaluation Setup

3.1 Environment

All experiments were conducted on a macOS 25.5 host with Python 3.11 and Java 21 (for SANY and TLC 1.8.0). The SysMoBench was cloned from main repository and setup per README.md instructions. The claude-code and codex were used to refine prompts and execute transition-validation evaluation step.

3.2 Tools

SysMoBench [5] The four-stage TLA+ evaluation pipeline described in Section 2.3.

Claude Sonnet 4.5 [7] Used for Experiments comparisons. Queried via the Anthropic API.

GPT-5.5 [8] Used for Experiments comparisons. Queried via the OpenAI API with temperature 0.7.

3.3 Dataset: Essential Paxos

The reference implementation is paxos/essential.py from the cocagne/paxos repository (commit cf3b5a2, branch master) [6]. The repository contains five Python modules totalling 1,091 lines and as previously mentioned in Section 2 we use only essential.py (202 lines), which implements the core single-decree protocol.

Execution traces were captured by running four deterministic scenarios through the run.py harness: *happy path* (single proposer reaches consensus), *dueling proposers* (two proposers compete, one wins), *message loss* (some messages are dropped and retried), and *late message* (an out-of-order delivery is processed after the protocol has already quiesced). Each scenario emits a separate NDJSON trace file under artifacts/essential_paxos/traces/.

3.4 Deviations from the Original SysMoBench Setup

Essential Paxos is not among the eleven systems in the SysMoBench 1.0 release [5]. We added it as a twelfth task following the documented docs/add_new_system.md procedure, contributing the following artifacts that are absent in the original benchmark:

- tla_eval/tasks/essential_paxos/task.yaml: task descriptor naming the source repository, specModule, trace folder, TV configuration, and label-to-action map.

- tla_eval/tasks/essential_paxos/prompts_claude/ and tla_eval/tasks/essential_paxos/prompts_codex/: three prompt files in each folder: direct_call.txt (spec generation), phase2_config.txt (TLC .cfg generation), and phase3_invariant_implementation.txt (invariant module generation).

- data/invariant_templates/essential_paxos/invariants.yaml: four written safety invariants: Agreement, Validity, Stability, and PromiseMonotonic.

- scripts/harness/essential_paxos/run.py: One notable implementation detail: upstream essential.py uses a Python 2-era None comparison inside Learner.recv_accepted that raises a TypeError under Python 3. We introduced a TracedLearner wrapper that substitutes a sentinel ProposalID(-1, "") for None before the comparison. The protocol semantics are unchanged, but this means the harness instruments a lightly patched version of the upstream source rather than the unmodified file.

4 Correctness Specifications and Verification Results

This section evaluates the generated TLA+ specifications using the four-stage SysMoBench pipeline: syntax checking, runtime model checking, transition validation, and invariant verification. We first define the correctness properties and TLC configuration used throughout the evaluation, and then compare the behaviour of the two prompting strategies across three experiments: iterative prompt refinement, cross-model prompt transfer, and scaling under larger TLC configurations.

4.1 Correctness Properties

We evaluate four expert-written invariants provided by SysMoBench:

- **Agreement** (safety): at most one value is ever decided across all learners at all times.
- **Validity** (safety): any decided value was proposed by some proposer; consensus cannot fabricate values.
- **Stability** (safety): once an acceptor accepts value v at proposal ID n, it never accepts a different value at the same n.
- **PromiseMonotonic** (temporal): an acceptor's promised proposal ID is non-decreasing over time.

Each failure was diagnosed from TLC/SANY error messages and translated into an additional constraint in the prompt. The most impactful additions were:

Table 1: Prompt refinement iteration history (Claude Sonnet 4.5)

Run	Outcome	Root cause
1	FAIL	Learners not assigned in `.cfg`
2	FAIL	Unbounded `CHOOSE n \in Nat`
3	FAIL	`>` applied to model value
4	FAIL	Infinite state space
5	FAIL	SANY parse error
6	FAIL	SANY parse error
7	FAIL	TLC exception
8	PASS	All four metrics passing

4.2 Model Checking Parameters

TLC is configured with: `Acceptors` = {`a1, a2, a3`}, `Proposers` = {`p1, p2`}, `Learners` = {`l1`}, `Values` = {`v1, v2`}, `MaxBallot = 3`. The constant `NoValue` is represented as a TLC model value acting as an opaque sentinel. Deadlock checking was disabled (`CHECK_DEADLOCK FALSE`), since Paxos can terminate normally after reaching consensus. The temporal property `PromiseMonotonic` was specified using `PROPERTY` rather than `INVARIANT`. This is because it constrains transitions between states instead of describing a single-state condition.

4.3 Experiment 1: Self-Iteration Phase

This experiment displays the iterative prompt refinement process for both LLMs independently. For each model, we started from an initial prompt generated by asking the model to inspect the existing SysMoBench prompts and the Essential Paxos repository and create a similar one. We ran SysMoBench via API keys. Then we fed each SANY/TLC failure back into the prompt as an additional constraint and regenerated until all four SysMoBench stages passed. The goal is to document *what errors each model makes*, in *what order*, and what final results were achieved.

4.3.1 Claude Sonnet 4.5 — Iteration Trajectory

Table 1 summarizes the eight generation attempts and the root cause of each failure prior to the final passing run.

1. **Bound the proposal ID space.** Replacing `Nat \X Proposers` with `(0..MaxBallot) \X Proposers` eliminates the infinite state space that causes TLC to diverge.
2. **Ballot-only comparison.** TLC model values (opaque identifiers) do not support the `<` operator. Adding the explicit instruction: `ProposalIDLt(a, b) == a[1] < b[1]` and forbidding `a[2] < b[2]` eliminates runtime type errors.
3. **Flat variable declarations.** SANY rejects multi-line nested record types used to group per-role variables. Requiring one variable per line avoids this class of parse errors.

Table 2: Prompt refinement iteration history (Codex)

Run	Outcome	Root cause
1	FAIL	TLA+ did not compile
2	PARTIAL	TV/invariants failed
3	PARTIAL	Malformed invariants
4	PASS	All four metrics passing

4. **Inline quorum check.** Replacing `CHOOSE n \in Nat : \E Q ...` with `Cardinality(S) * 2 > Cardinality(Acceptors)` is necessary because TLC cannot enumerate infinite sets.

The two remaining failure classes are: (1) a double-assignment bug in `HandleAccepted` that makes the quorum-detection branch logically unreachable (causing the 29% pass rate on that action), and (2) ballot-only comparison in `ProposalIDLt` that disagrees with the Python implementation's lexicographic tiebreaker in dueling-proposer scenarios (3 of the 14 HandlePrepare failures, 1 HandlePromise failure).

Table 3 shows the per-action transition validation breakdown.

4.3.2 Codex (Extra-High) and GPT-5.5

Table 2 summarizes the four refinement iterations required before all SysMoBench stages fully passed.

The first generated specifications failed to compile because the prompt allowed several TLC-incompatible constructs, including unbounded ballot domains, tuple/model-value comparisons, and cfg bindings for tuple sentinels. These issues were resolved by introducing finite proposal-ID spaces, numeric proposer identifiers, explicit ordering helpers, and a ban on comparisons involving opaque TLC model values.

After compilation succeeded, the remaining failures were primarily caused by mismatches between the generated state abstraction and the execution traces used during transition validation. The most impactful prompt changes were:

1. **Move value selection into `Prepare`.** The Python helper `set_proposal(value)` is not explicitly traced by the harness. Folding value selection into the `Prepare` action aligned the generated transitions with the observed execution traces.

2. **Derive learner state from `msgs`.** Learner quorum detection was rewritten to derive accepted messages directly from the global message set rather than hidden learner bookkeeping variables. This allowed non-decision deliveries to appear as valid steps during transition validation.

3. **Restrict invariant syntax to TLC-safe formulas.** The initial invariant translation generated malformed temporal formulas and invalid filtered set comprehensions. The prompt was therefore constrained to emit `PromiseMonotonic` as a single `[][...]_vars` property and to express `Validity` using simple set membership.

The final specification passed all SysMoBench stages used in our evaluation, including syntax checking, runtime checking, transition validation, and all four invariants. Table 3 shows the final per-action results.

4.3.3 Comparison of Iteration Trajectories

Claude required 8 iterations before all SysMoBench stages passed while GPT-5.5 required 4 iterations. Both models independently encountered the same class of finite-domain errors: both needed explicit `ProposalIDLt` helpers, bounded ballot sets, and majority checks expressed as cardinality inequalities over finite sets. Claude was uniquely affected by parse errors from multi-line nested record types and by the SANY forward-reference restriction, which caused operators referencing variables to be rejected when declared before the `VARIABLES` block. GPT-5.5 was uniquely affected by message union helpers being wrongly counted as protocol actions (causing action-decomposition partial failures), and by temporal formula shape errors in the `PromiseMonotonic` invariant (`[][][...]_vars` instead of the correct `[][...]_vars`).

The final prompts differ substantially in sentinel strategy: Prompt_C uses `NoProposal` as a TLC model value declared as a `CONSTANT`, while Prompt_X defines `NoProposal == ⟨⟨0, 0⟩⟩` inside the module, avoiding a cfg-level binding of a tuple expression. The Codex prompt also provides a more detailed learner quorum helper (`LatestAcceptedBy`) derived from `msgs`, which is why it achieves 100% TV while the simpler Claude prompt reaches 78.7%.

Table 3: Experiment 2: SysMoBench results across (model, prompt) cells

Model	Prompt	Compile	Runtime	TV rate	Invariants
Claude Sonnet 4.5	Prompt_C (Claude)	PASS	PASS	78.7% (48/61)	4/4
Claude Sonnet 4.5	Prompt_X (Codex)	FAIL	—	—	—
GPT-5 / Codex	Prompt_C (Claude)	FAIL	—	—	—
GPT-5 / Codex	Prompt_X (Codex)	PASS	PASS	100% (61/61)	4/4

4.4 Experiment 2: Comparing Prompts Across Models

Experiment 2 evaluates whether the refined prompts from Experiment 1 transfer across models. We take the final Claude prompt (Prompt_C) and the final Codex prompt (Prompt_X), run each in a fresh session on both models, and evaluate the resulting specifications using the full SysMoBench pipeline. Table 3 summarizes the results.

The outcome is strongly model-specific. Both models pass when paired with their own refined prompts, while both cross-model combinations fail during compilation.

For the Claude + Prompt_X configuration, SANY reported two *Unknown operator* errors near the start of the module. Claude consistently emitted the `NextProposalNum` helper before the `VARIABLES` declaration even though it referenced `proposalId`, which SANY rejects as a forward reference. This suggests that Prompt_X implicitly relied on GPT-5's generation order without explicitly constraining helper placement.

The GPT-5 + Prompt_C configuration also failed to compile. The shorter, constraint-heavy style of the Claude-oriented prompt, combined with its use of `NoProposal` as a TLC model value rather than a tuple alias, conflicted with GPT-5's generation patterns and produced invalid TLA+ output.

The strongest overall result is achieved by GPT-5/Codex with Prompt_X, reaching 100% transition validation and passing all four invariants.

4.5 Experiment 3: Configuration Scaling

In this experiment we evaluate both generated specifications under three TLC configurations that scale the number of participants and the ballot bound. Since both models only succeeded with their own refined prompts, we evaluate both final specifications rather than selecting a single prompt/model combination. Both specs are checked under identical configurations with a strict 90-second wall-clock cap; TLC is stopped after 90 seconds and the explored state count is recorded. This cap makes the two specifications directly comparable regardless of their final state-space size.

Three findings stand out. First, the Claude specification is substantially cheaper to verify: it completes both Small and Baseline within the time limit, whereas the GPT-5 specification completes only Small. Second, the GPT-5 specification explores a dramatically larger state space despite identical `.cfg` parameters, reaching roughly 1.86M states at Small versus 1,217 for Claude, and more than 3.7M states at Baseline before timing out. This is caused primarily by the GPT-5 specification's `HandlePromise` action, which includes an unconstrained stutter branch (`UNCHANGED vars`) representing message loss. For every Promise message, TLC must therefore explore both a processed and ignored execution path, causing exponential branching. These scaling trends are consistent with SysMoBench's findings on etcd Raft and raftkvs [5].

Finally, `TypeOK` holds in all completed runs for both specifications, confirming type safety at every evaluated scale. The TV score (78.7% for Claude, 100% for GPT-5) remains unchanged across configurations because transition validation is evaluated against fixed execution-trace windows rather than TLC's explored state space.

Table 4: Configuration parameters for Experiment 3

Configuration	Small	Baseline	Large
Acceptors	3	3	5
Proposers	1	2	3
Learners	1	1	1
MaxBallot	2	3	3
Values	2	2	2

Table 5: Experiment 3: TLC model checking results under 90 s cap (both specs)

	Claude + Prompt_C			GPT-5 + Prompt_X		
Config	States	Distinct	Done?	States	Distinct	Done?
Small	1,217	244	Yes (<1 s)	1,857,794	72,764	Yes (∼36 s)
Baseline	1,502,755	222,984	Yes (∼26 s)	>3,680,568	>397,309	No
Large	>3,696,573	>779,841	No	>2,548,004	>441,963	No

5 Related Work

TLA+ verification of consensus protocols. Lamport himself has published TLA+ specifications of the Paxos consensus algorithm [9] and various Paxos variants such as Fast Paxos [10]. These hand-written specifications serve as the gold standard for correctness and are the basis for the safety invariants we use in this study. Howard and Mortier [11] provide a comparative formal analysis of Raft and Paxos consensus protocols in TLA+, demonstrating that even subtle implementation differences lead to distinct state machines. Our work differs in that the specification is *generated*, not hand-written, and our goal is to characterize how well LLMs can recover a reference specification from its Python implementation. Unlike Lamport's own TLA+ specs [9], which cover the full protocol family, we restrict our model to the `essential.py` core, making the comparison between prompt strategies cleaner by holding protocol complexity constant.

Raft and industrial consensus in TLA+. The Raft consensus protocol [12] ships with an official TLA+ specification that has been model-checked against safety invariants similar to those we use. The SysMoBench benchmark itself [5] includes etcd Raft and Redis Raft as tasks, providing directly comparable LLM generation results. Our Essential Paxos task is simpler than Raft, making it a more controlled setting for studying prompt-engineering effects and comparing the results with the official specification.

LLM-assisted formal specification. Several recent works explore using LLMs to generate or repair formal specifications. Endres et al. [13] investigate the *nl2postcond* task, in which LLMs translate natural-language program intent into formal post-conditions. They report that generated specifications are frequently syntactically correct and often useful for bug detection, but that correctness does not guarantee completeness with respect to developer intent. [13] Our findings exhibit a similar phenomenon in a different setting. Generated TLA+ specifications often compile, model-check, and satisfy structural correctness criteria, yet still diverge from the reference implementation's behaviour under transition validation. This suggests that semantic alignment remains a central challenge across both assertion generation and executable formal-specification synthesis. The closest precedent to our work is still the SysMoBench paper itself [5], which evaluates a range of models (GPT-4o, Claude Opus, DeepSeek-R1) on eleven systems using the same four-stage pipeline we adopt. SysMoBench reports that no model achieves a top-tier score on all systems, and that transition validation is the hardest stage for every model evaluated. Our contribution extends this evaluation to Essential Paxos and studies whether iterative prompt refinement, rather than model capability alone, is the decisive factor.

Prompt engineering for code generation. Wei et al. [14] show that chain-of-thought prompting improves multi-step reasoning in LLMs. Subsequent work has applied structured and feedback-driven prompting to program synthesis. Chen et al. [15] propose *self-debugging*, in which a model iteratively revises its generated code in response to execution results, and the broader *self-repair* paradigm it exemplifies has since been studied across models and benchmarks [16]. Our iterative refinement procedure is an instance of this paradigm applied to TLA+ generation, with SANY and TLC replacing the compiler and test suite. Compared to automated self-repair, our approach, or rather in this case chosen LLM, also takes the feedback and automatically translates it into prompt constraints rather than injected manually by us (developers). Even though the process happens automatically, LLM changes history allows us to trace exactly which constraints were necessary for each failure mode, producing the per-iteration

analysis in Table 1 and Table 2.

Benchmarks for AI-generated specifications. Beyond SysMoBench, OSVBENCH [17] benchmarks LLMs on specification generation for operating system verification, finding that current models can produce compilable specifications but frequently violate subtle semantic constraints, which is a finding consistent with our observation that syntax and runtime correctness are necessary but insufficient for faithful specifications. Music et al. [18] address the related problem of validating execution traces of distributed programs against TLA+ specifications, identifying practical challenges in bridging implementation-level events to specification-level state changes that are directly relevant to SysMoBench's transition-validation stage and to the custom TV runner we developed. The key novelty of SysMoBench, and therefore of our study, is the use of *actual execution traces* from the reference system as the ground truth for transition validation, rather than relying solely on human-written invariants.

6 Limitations

7 Statement of GenAI Use

This project uses GenAI in two ways, consistent with the course policy:

1. **Specification generation.** The TLA+ specification (`EssentialPaxos.tla`) was generated by Claude Sonnet 4.5 (Anthropic) and Chat GPT 5 (Open AI) via the SysMoBench `direct_call` pipeline. The specification was *not* hand-edited. All improvements were made exclusively by modifying the prompt.
2. **Report drafting and tooling assistance.** Both Claude Code (Anthropic) and Codex (Open AI) were used to assist with Python scripting (the custom TV runner), debugging TLC error messages, and drafting sections of this report. All technical claims and report content were verified independently by the authors.

References

[1] S. Chand, Y. A. Liu, and S. D. Stoller. Formal Verification of Multi-Paxos for Distributed Consensus. CoRR, abs/1606.01387, 2016. https://arxiv.org/abs/1606.01387.

[2] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. Available at: https://lamport.org/tla/book.html

[3] Y. Yu, P. Manolios, and L. Lamport. Model Checking TLA+ Specifications. *Correct Hardware Design and Verification Methods (CHARME'99)*, LNCS 1703, pages 54–66, Springer, 1999. Available at: https://lamport.org/pubs/yuanyu-model-checking.pdf

[4] M. Chen et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021. Available at: https://arxiv.org/abs/2107.03374

[5] Q. Cheng et al. SysMoBench: Evaluating AI on Formally Modeling Complex Real-World Systems. *ICLR*, 2026. https://arxiv.org/abs/2509.23130

[6] [Author(s) of Essential Paxos Python implementation]. Essential Paxos. https://github.com/cocagne/paxos/blob/master/paxos/essential.py

[7] Anthropic. Claude Sonnet 4.5. https://www.anthropic.com, 2025.

[8] OpenAI. GPT-5: A Unified System for Advanced Reasoning and Multimodal Intelligence. https://openai.com/index/introducing-gpt-5, 2025.

[9] L. Lamport. The Paxos Algorithm or How to Win a Turing Award. Available at: https://lamport.azurewebsites.net/tla/paxos-algorithm.html

[10] L. Lamport. Fast Paxos. *Distributed Computing*, 19(2):79–103, 2006. Available at: https://doi.org/10.1007/s00446-006-0005-x

[11] H. Howard and R. Mortier. Paxos vs Raft: Have we reached consensus on distributed consensus? *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC)*, 2020. Available at: https://doi.org/10.1145/3380787.3393681

[12] D. Ongaro. Consensus: Bridging Theory and Practice. PhD thesis, Stanford University, 2014. Available at: https://ongardie.net/var/blurbs/pubs/dissertation.pdf

[13] M. Endres et al. Can LLMs Transform Natural Language Intent into Formal Method Postconditions? *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):Article 84, 2024. https://doi.org/10.1145/3660791

[14] J. Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS*, 2022. Available at: https://arxiv.org/abs/2201.11903

[15] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching Large Language Models to Self-Debug. *ICLR*, 2024. arXiv:2304.05128. Available at: https://arxiv.org/abs/2304.05128

[16] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama. Is Self-Repair a Silver Bullet for Code Generation? *ICLR*, 2024. arXiv:2306.09896. Available at: https://arxiv.org/abs/2306.09896

[17] X. Wang et al. OSVBENCH: Benchmarking LLMs on Specification Generation Tasks for Operating System Verification. *AAAI Conference on Artificial Intelligence*, 2026. https://arxiv.org/abs/2504.20964

[18] H. Music, O. Gurov, and L. Lamport. Validating Traces of Distributed Programs Against TLA+ Specifications. *Software Engineering and Formal Methods (SEFM)*, 2024. https://inria.hal.science/hal-04813639